



Linux Fundamentals

0%



Section 27 / 30

Firewall Setup

The primary goal of firewalls is to provide a security mechanism for controlling and monitoring network traffic between different network segments, such as internal and external networks or different network zones. Firewalls play a crucial role in protecting computer networks from unauthorized access, malicious traffic, and other security threats. Linux, being a popular operating system used in servers and other network devices, provides built-in firewall capabilities that can be used to control network traffic. In other words, they can filter incoming and outgoing traffic based on pre-defined rules, protocols, ports, and other criteria to prevent unauthorized access and mitigate security threats. The specific goal of a firewall implementation can vary depending on the specific needs of the organization, such as ensuring the confidentiality, integrity, and availability of network resources.

An example from the history of Linux firewalls is the development of the iptables tool, which replaced the earlier ipchains and ipfwadm tools. The iptables utility was first introduced in the Linux 2.4 kernel in 2000 and provided a flexible and efficient mechanism for filtering network traffic. iptables became the de facto standard firewall solution for Linux systems, and it has been widely adopted by many organizations and users.

The iptables utility provided a simple yet powerful command-line interface for configuring firewall rules, which could be used to filter traffic based on various criteria such as IP addresses, ports, protocols, and more. iptables was designed to be highly customizable and could be used to create complex firewall rulesets that could protect against various security threats such as denial-of-service (DoS) attacks, port scans, and network intrusion attempts.

In Linux, the firewall functionality is typically implemented using the Netfilter framework, which is an integral part of the kernel. Netfilter provides a set of hooks that can be used to intercept and modify network traffic as it passes through the system. The iptables utility is commonly used to configure the firewall rules on Linux systems.

Iptables

The iptables utility provides a flexible set of rules for filtering network traffic based on various criteria such as source and destination IP addresses, port numbers, protocols, and more. There also exist other solutions like nftables, ufw, and firewalld. **Nftables** provides a more modern syntax and improved performance over iptables. However, the syntax of nftables rules is not compatible with iptables, so migration to nftables requires some effort. **UFW** stands for “Uncomplicated Firewall” and provides a simple and user-friendly interface for configuring firewall rules. UFW is built on top of the iptables framework like nftables and provides an easier way to manage firewall rules. Finally, Firewalld provides a dynamic and flexible firewall solution that can be used to manage complex firewall configurations, and it supports a rich set of rules for filtering network traffic and can be used to create custom firewall zones and services. It consists of several components that work together to provide a flexible and powerful firewall solution. The main components of iptables are:

Component	Description
Tables	Tables are used to organize and categorize firewall rules.
Chains	Chains are used to group a set of firewall rules applied to a specific type of network traffic.
Rules	Rules define the criteria for filtering network traffic and the actions to take for packets that match the criteria.
Matches	Matches are used to match specific criteria for filtering network traffic, such as source or destination IP addresses, ports, protocols, and more.
Targets	Targets specify the action for packets that match a specific rule. For example, targets can be used to accept, drop, or reject packets or modify the packets in another way.

Tables

When working with firewalls on Linux systems, it is important to understand how tables work in iptables. Tables in iptables are used to categorize and organize firewall rules based on the type of traffic that they are designed to handle. These tables are used to organize and categorize firewall rules. Each table is responsible for performing a specific set of tasks.

Table Name	Description	Built-in Chains
<code>filter</code>	Used to filter network traffic based on IP addresses, ports, and protocols.	INPUT, OUTPUT, FORWARD
<code>nat</code>	Used to modify the source or destination IP addresses of network packets.	PREROUTING, POSTROUTING
<code>mangle</code>	Used to modify the header fields of network packets.	PREROUTING, OUTPUT, INPUT, FORWARD, POSTROUTING

In addition to the built-in tables, iptables provides a fourth table called the raw table, which is used to configure special packet processing options. The raw table contains two built-in chains: PREROUTING and OUTPUT.

Chains

In iptables, chains organize rules that define how network traffic should be filtered or modified. There are two types of chains in iptables:

- Built-in chains
- User-defined chains

The built-in chains are pre-defined and automatically created when a table is created. Each table has a different set of built-in chains. For example, the filter table has three built-in chains:

- INPUT

- OUTPUT
- FORWARD

These chains are used to filter incoming and outgoing network traffic, as well as traffic that is being forwarded between different network interfaces. The nat table has two built-in chains:

- PREROUTING
- POSTROUTING

The PREROUTING chain is used to modify the destination IP address of incoming packets before the routing table processes them. The POSTROUTING chain is used to modify the source IP address of outgoing packets after the routing table has processed them. The mangle table has five built-in chains:

- PREROUTING
- OUTPUT
- INPUT
- FORWARD
- POSTROUTING

These chains are used to modify the header fields of incoming and outgoing packets and packets being processed by the corresponding chains.

User-defined chains can simplify rule management by grouping firewall rules based on specific criteria, such as source IP address, destination port, or protocol. They can be added to any of the three main tables. For example, if an organization has multiple web servers that all require similar firewall rules, the rules for each server could be grouped in a user-defined chain.

Another example is when a user-defined chain could filter traffic destined for a specific port, such as port 80 (HTTP). The user could then add rules to this chain that specifically filter traffic destined for port 80.

Rules and Targets

Iptables rules are used to define the criteria for filtering network traffic and the actions to take for packets that match the criteria. Rules are added to chains using the `-A` option followed by the chain name, and they can be modified or deleted using various other options.

Each rule consists of a set of criteria or matches and a target specifying the action for packets that match the criteria. The criteria or matches match specific fields in the IP header, such as the source or destination IP address, protocol, source, destination port number, and more. The target specifies the action for packets that match the criteria. They specify the action to take for packets that match a specific rule. For example, targets can accept, drop, reject, or modify the packets. Some of the common targets used in iptables rules include the following:

Target Name	Description
<code>ACCEPT</code>	Allows the packet to pass through the firewall and continue to its destination
<code>DROP</code>	Drops the packet, effectively blocking it from passing through the firewall
<code>REJECT</code>	Drops the packet and sends an error message back to the source address, notifying them that the packet was blocked

Target Name	Description
LOG	Logs the packet information to the system log
SNAT	Modifies the source IP address of the packet, typically used for Network Address Translation (NAT) to translate private IP addresses to public IP addresses
DNAT	Modifies the destination IP address of the packet, typically used for NAT to forward traffic from one IP address to another
MASQUERADE	Similar to SNAT but used when the source IP address is not fixed, such as in a dynamic IP address scenario
REDIRECT	Redirects packets to another port or IP address
MARK	Adds or modifies the Netfilter mark value of the packet, which can be used for advanced routing or other purposes

Let us illustrate a rule and consider that we want to add a new entry to the INPUT chain that allows incoming TCP traffic on port 22 (SSH) to be accepted. The command for that would look like the following:

```
athane@htb[/htb]$ sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
```

shellsession

Matches

Matches are used to specify the criteria that determine whether a firewall rule should be applied to a particular packet or connection. Matches are used to match specific characteristics of network traffic, such as the source or destination IP address, protocol, port number, and more.

Match Name	Description
<code>-p</code> or <code>--protocol</code>	Specifies the protocol to match (e.g. tcp, udp, icmp)
<code>--dport</code>	Specifies the destination port to match
<code>--sport</code>	Specifies the source port to match
<code>-s</code> or <code>--source</code>	Specifies the source IP address to match
<code>-d</code> or <code>--destination</code>	Specifies the destination IP address to match
<code>-m state</code>	Matches the state of a connection (e.g. NEW, ESTABLISHED, RELATED)
<code>-m multiport</code>	Matches multiple ports or port ranges
<code>-m tcp</code>	Matches TCP packets and includes additional TCP-specific options
<code>-m udp</code>	Matches UDP packets and includes additional UDP-specific options
<code>-m string</code>	Matches packets that contain a specific string

Match Name	Description
<code>-m limit</code>	Matches packets at a specified rate limit
<code>-m conntrack</code>	Matches packets based on their connection tracking information
<code>-m mark</code>	Matches packets based on their Netfilter mark value
<code>-m mac</code>	Matches packets based on their MAC address
<code>-m iprange</code>	Matches packets based on a range of IP addresses

In general, matches are specified using the '-m' option in iptables. For example, the following command adds a rule to the 'INPUT' chain in the 'filter' table that matches incoming TCP traffic on port 80:

```
athane@htb[/htb]$ sudo iptables -A INPUT -p tcp -m tcp --dport 80 -j ACCEPT
```

shellsession

This example rule matches incoming TCP traffic (`-p tcp`) on port 80 (`--dport 80`) and jumps to the accept target (`-j ACCEPT`) if the match is successful.

1. Launch a web server on TCP/8080 port on your target and use iptables to block incoming traffic on that port.

2. Change iptables rules to allow incoming traffic on the TCP/8080 port.
3. Block traffic from a specific IP address.
4. Allow traffic from a specific IP address.
5. Block traffic based on protocol.
6. Allow traffic based on protocol.
7. Create a new chain.
8. Forward traffic to a specific chain.
9. Delete a specific rule.
10. List all existing rules.

Connect to HTB



27 / 30 Sections



Mark Complete & Next